

校园体育赛事助手 - 测试报告

一、后端 API 功能测试

测试内容：使用 Jest + supertest 对后端 Express 接口进行功能回归测试。覆盖健康检查、赛程列表、积分榜、球员榜四个核心接口，验证 HTTP 状态码、响应结构、关键字段（如 `home_team_name`、`away_team_name`、`team_name`、`player_name`）的正确性，确保接口返回的数据结构完整且符合前端需求。

编写方法：在 `PKU_CSports_backend/tests/api.test.js` 中，使用 `supertest` 直接挂载 Express 应用（不启动真实端口），通过 `request(app).get()` 发起请求，用 Jest 的 `expect` 断言状态码、`success` 字段和数据结构。测试依赖 `scripts/seed.js` 预置的测试数据，确保可重复执行。

所用框架：Jest（测试运行器与断言库）、supertest（HTTP 断言库）。

最终结果：4 个用例全部通过，验证了接口返回正确状态码、数据结构完整，以及联表查询返回球队/球员名称字段，为前端数据展示提供了可靠保障。

二、后端认证接口功能测试

测试内容：对登录接口 `/api/auth/login` 进行全面的测试，包括参数验证（空用户名/密码）、错误密码处理、成功登录场景，以及不同角色用户（普通用户、协会用户）的登录验证。确保认证流程的健壮性和安全性。

编写方法：在 `PKU_CSports_backend/tests/api/auth.test.js` 中，使用 `supertest` 发送 POST 请求，测试各种输入场景。验证成功登录时返回的 `token` 和用户信息结构，确保角色字段正确返回。

所用框架：Jest、supertest。

最终结果：4 个测试用例全部通过，验证了登录接口的参数校验、错误处理和成功场景，确保不同角色用户都能正确登录并获得相应权限。

三、后端权限控制测试

测试内容：对赛程接口的权限控制进行测试，验证未认证请求、普通用户和协会用户对敏感操作（如更新比分）的访问权限。确保系统的安全性和角色权限的正确实现。

编写方法：在 `PKU_CSports_backend/tests/api/matches-auth.test.js` 中，使用 JWT 生成不同角色的 token，测试 `/api/matches/:id/result` 接口在不同权限下的响应。验证未认证返回 401，普通用户返回 403，协会用户允许访问。

所用框架：Jest、supertest、jsonwebtoken。

最终结果：3 个测试用例全部通过，验证了权限中间件正确工作，普通用户无法执行协会专属操作，系统安全性得到保障。

四、后端单元测试 - 密码工具函数

测试内容：对密码工具函数 `hashPassword` 和 `comparePassword` 进行单元测试，验证密码哈希的唯一性（每次生成不同的哈希值）、密码验证的正确性（正确密码通过、错误密码拒绝），以及边界情况处理（空密码）。

编写方法：在 `PKU_CSports_backend/tests/unit/password.test.js` 中，直接调用工具函数，测试密码哈希和验证逻辑。验证 `bcrypt` 生成的哈希值长度和唯一性，确保密码安全存储和验证机制正确。

所用框架：Jest。

最终结果：3 个测试用例全部通过，验证了密码工具函数能够正确生成唯一哈希值、准确验证密码，为系统安全提供了基础保障。

五、后端单元测试 - 认证中间件

测试内容：对认证中间件 `authMiddleware` 和角色权限中间件 `allowRoles` 进行单元测试，验证 JWT token 解析、无效 token 拒绝、用户信息附加，以及基于角色的访问控制。

编写方法：在 `PKU_CSports_backend/tests/unit/auth.test.js` 中，使用 Jest 的 `mock` 功能模拟请求和响应对象，测试中间件的各种场景。验证无 token、无效 token、有效 token 的处理，以及角色权限的校验逻辑。

所用框架：Jest、`jsonwebtoken`。

最终结果：5 个测试用例全部通过，验证了认证中间件能够正确解析 JWT、拒绝无效请求、附加用户信息，角色权限控制机制工作正常。

六、后端集成测试

测试内容：对赛程模块的完整业务流程进行集成测试，模拟真实用户操作流程：查看赛程列表 → 查看赛程详情 → 协会用户更新比分。验证多个接口协同工作的正确性。

编写方法：在 `PKU_CSports_backend/tests/integration/match-flow.test.js` 中，使用 `supertest` 依次调用多个接口，形成完整的业务流程。通过 `beforeAll` 准备协会用户 token，在测试中验证整个流程的连贯性和正确性。

所用框架：Jest、`supertest`、`jsonwebtoken`。

最终结果：测试通过，验证了系统能够支持完整的业务流程，接口之间的数据传递和权限控制正确，用户体验流畅。

七、后端性能测试 - 积分榜接口

测试内容：使用 `k6` 对积分榜和球员榜接口进行性能测试，模拟 15 个虚拟用户持续 30 秒的并发请求，验证接口在负载下的响应时间和成功率。

编写方法：在 `PKU_CSports_backend/tests/perf/standings.k6.js` 中，配置 `vus: 15` 和 `duration: '30s'`，设置性能阈值 `http_req_duration: ['p(95)<500']`（95% 的请求响应时间应小于 500 毫秒）。测试积分榜和球员榜两个接口，验证状态码和响应体内容。

所用框架：`k6`（性能与压力测试工具）。

最终结果：测试通过，95% 的请求响应时间在 500 毫秒以内，接口在 15 并发下表现稳定，满足性能要求。

八、后端性能测试 - 新闻接口

测试内容：对新闻接口（快讯和战报）进行性能测试，模拟 10 个虚拟用户持续 30 秒的并发请求，验证新闻列表接口的响应性能。

编写方法：在 `PKU_CSports_backend/tests/perf/news.k6.js` 中，配置并发参数和性能阈值，分别测试快讯和战报两个接口，验证响应状态码和数据结构。

所用框架：k6。

最终结果：测试通过，新闻接口在 10 并发下响应时间在 400 毫秒以内，性能表现良好。

九、后端压力测试

测试内容：对系统进行阶梯式压力测试，模拟负载从 20 用户逐步增加到 100 用户，然后逐步降回 0，验证系统在高负载下的稳定性和性能表现。

编写方法：在 `PKU_CSports_backend/tests/perf/stress.k6.js` 中，使用 k6 的 `stages` 配置阶梯式负载增长：30 秒内增加到 20 用户，1 分钟内增加到 50 用户，30 秒内增加到 100 用户，保持 1 分钟后逐步降回 0。随机访问赛程、积分榜、新闻三个接口，设置性能阈值（95% 请求 < 1 秒，错误率 < 5%）。

所用框架：k6、自定义错误率指标。

最终结果：测试通过，系统在 100 并发下仍能保持稳定，95% 的请求响应时间在 1 秒以内，错误率低于 5%，系统具备良好的抗压能力。

十、前端单元测试 - 请求工具函数

测试内容：使用 Vitest 对前端工具函数 `utils/request.js` 进行单元测试，验证请求函数能自动拼接 `baseURL`（将相对路径转为完整 URL），并能从本地存储读取 token 并添加到请求头 `Authorization` 中。

编写方法：在 `PKU_CSports_frontend/tests/unit/request.spec.ts` 中，使用 `vi.fn()` mock `uni.request` 和 `uni.getStorageSync`，模拟成功回调和 token 返回值。调用 `request()` 后，通过 `expect(uni.request).toHaveBeenCalledTimes(1)` 验证调用次数，并检查调用参数中的 `url` 和 `header.Authorization` 是否符合预期。通过 `tests/setup/global.js` 在测试环境中注入全局 `uni` 对象，避免运行时错误。

所用框架：Vitest（测试运行器）、@vue/test-utils（Vue 组件测试工具，本用例主要用于工具函数测试）。

最终结果：测试通过，验证了请求工具函数能正确拼接 URL 和添加认证头，为前端网络请求层提供了基础保障。

十一、前端单元测试 - 通用工具函数

测试内容：对前端通用工具函数进行单元测试，包括日期格式化、数组过滤、对象属性提取等常用功能，确保数据处理逻辑的正确性。

编写方法：在 `PKU_CSports_frontend/tests/unit/utils.spec.ts` 中，测试日期字符串的切片操作、数组过滤逻辑、对象属性的回退处理等，验证这些工具函数能够正确处理各种输入场景。

所用框架：Vitest。

最终结果：测试通过，验证了通用工具函数能够正确处理数据，为前端业务逻辑提供了可靠支持。

十二、前端界面端到端测试 - 赛事页面

测试内容：使用 Cypress 对赛事页面进行端到端测试，验证页面能正确加载、发起 API 请求、渲染赛程卡片，并显示球队名称和"vs"标识。

编写方法：在 `PKU_CSports_frontend/tests/e2e/matches.cy.ts` 中，使用 `cy.intercept()` 拦截 `/api/matches` 接口并返回固定数据，避免依赖后端服务稳定性。通过 `cy.visit()` 访问页面，使用 `cy.contains()` 和 `cy.get()` 定位元素，通过 `cy.wait('@getMatches')` 等待接口响应，最后断言 `.match-card` 元素存在且包含预期的球队名称。设置 8 秒超时以适应页面加载时间。

所用框架：Cypress（端到端测试框架）。

最终结果：测试通过，验证了前端页面能正确加载数据并渲染 UI，确保了用户可见功能的正确性。

十三、前端界面端到端测试 - 数据中心页面

测试内容：对数据中心页面进行端到端测试，验证页面能够正确切换联赛标签和数据类型标签（球员、球队、积分），并正确展示相应的数据列表。

编写方法：在 `PKU_CSports_frontend/tests/e2e/data.cy.ts` 中，使用 `cy.intercept()` 分别拦截球员榜和积分榜接口，返回模拟数据。访问页面后，点击"积分"标签，等待接口响应，验证数据列表正确渲染。

所用框架：Cypress。

最终结果：测试通过，验证了数据中心页面的标签切换功能正常，数据展示正确，用户体验良好。

十四、前端界面端到端测试 - 工具页面权限

测试内容：对工具页面进行基于角色的权限测试，验证普通用户和协会用户看到不同的 UI 元素（发布按钮的显示/隐藏），确保前端权限控制正确实现。

编写方法：在 `PKU_CSports_frontend/tests/e2e/tools.cy.ts` 中，使用 `cy.window().then()` 操作 `localStorage`，模拟不同角色的用户登录状态。分别测试普通用户和协会用户访问工具页面，验证发布按钮的存在与否。使用 `cy.clearLocalStorage()` 确保测试隔离。

所用框架：Cypress。

最终结果：测试通过，验证了前端能够根据用户角色正确显示/隐藏功能按钮，权限控制机制工作正常，提升了系统的安全性和用户体验。

十五、前端性能测试 - 接口性能

测试内容：使用 k6 对前端调用的赛程接口进行性能测试，模拟 10 个虚拟用户持续 30 秒的并发请求，验证接口在负载下的响应时间和成功率。

编写方法：在 `PKU_CSports_frontend/tests/perf/matches.k6.js` 中，配置并发参数和性能阈值，测试赛程接口的响应时间和数据完整性。

所用框架：k6。

最终结果：测试通过，接口在 10 并发下响应时间在 600 毫秒以内，性能表现良好。

十六、前端性能测试 - 页面性能审计

测试内容：使用 Lighthouse + Puppeteer 对前端 H5 页面进行性能评估，测量页面的性能分数、最大内容绘制时间（LCP）和可交互时间（TTI）等核心 Web 性能指标。

编写方法：在 `PKU_CSports_frontend/tests/perf/lighthouse.js` 中，使用 Puppeteer 启动无头浏览器，通过 Lighthouse 对目标 URL（默认 `http://localhost:4173/#/pages/matches/matches`）进行性能审计。配置输出格式为 JSON，从结果中提取性能分数、LCP 和 TTI 的显示值并输出到控制台。

所用框架：Lighthouse（性能审计工具）、Puppeteer（无头浏览器控制）。

最终结果：成功获取性能指标，可用于评估页面加载速度和用户体验，为性能优化提供数据支持。

测试总结

本次测试覆盖了后端和前端两个层面，包括：

测试类型分布

- **单元测试：**后端密码工具、认证中间件；前端请求工具、通用工具函数（共 8 个测试文件）
- **功能测试：**后端 API 接口、认证接口、权限控制（共 3 个测试文件）
- **集成测试：**后端完整业务流程（1 个测试文件）
- **界面测试：**前端赛事页、数据中心页、工具页权限（共 3 个 E2E 测试文件）
- **性能测试：**后端积分榜、新闻接口；前端接口性能（共 3 个性能测试文件）
- **压力测试：**后端阶梯式负载测试（1 个压力测试文件）

测试统计

- **后端测试用例总数：**约 20+ 个测试用例
- **前端测试用例总数：**约 10+ 个测试用例
- **测试覆盖范围：**核心业务功能、权限控制、性能表现、用户体验

测试成果

所有测试用例均通过，验证了系统在以下方面的正确性：

1. **功能正确性：**接口返回正确的数据结构和状态码
2. **安全性：**密码加密、JWT 认证、角色权限控制
3. **性能表现：**接口响应时间满足要求，系统具备良好的抗压能力
4. **用户体验：**前端页面正确渲染，权限控制正确实现

5. 系统稳定性：在 100 并发下仍能保持稳定运行

测试框架的搭建为后续持续集成和回归测试奠定了基础，有助于保障代码质量和系统稳定性，为项目的长期维护和迭代提供了可靠保障。